LAB MANUAL

K.R. MANGALAM UNIVERSITY

SOET

(School of Engineering & Technology)

B.TECH CSE (CORE)

Database & Management System

Submitted by-

Dolly

Roll no - 2401010196

Submitted to-

Mansi Kajal

Experiment-1

Aim
To design an Entity–Relationship (ER) diagram for a company database system and define the relationship types between entities.

Objectives

- To identify entities and their attributes in a company database
- To represent relationships among entities
- To define relationship types such as one-to-one, one-to-many, and many-to-many
- To represent the number of hours an employee works on a project

Entities and Attributes

1. Staff (represents organization staff members)

- StaffID (Primary Key)
- FullName
- Residence
- Gender
- Pay
- SupervisorID
- DivisionID

2. Division (organization divisions)

- DivID (Primary Key)
- DivName
- LeadStaffID
- LeadStartDate

3. Task (tasks assigned in the organization)

- TaskID (Primary Key)
- TaskName
- TaskSite
- DivID

4. DivSite (division sites)

- SiteName
- DivID

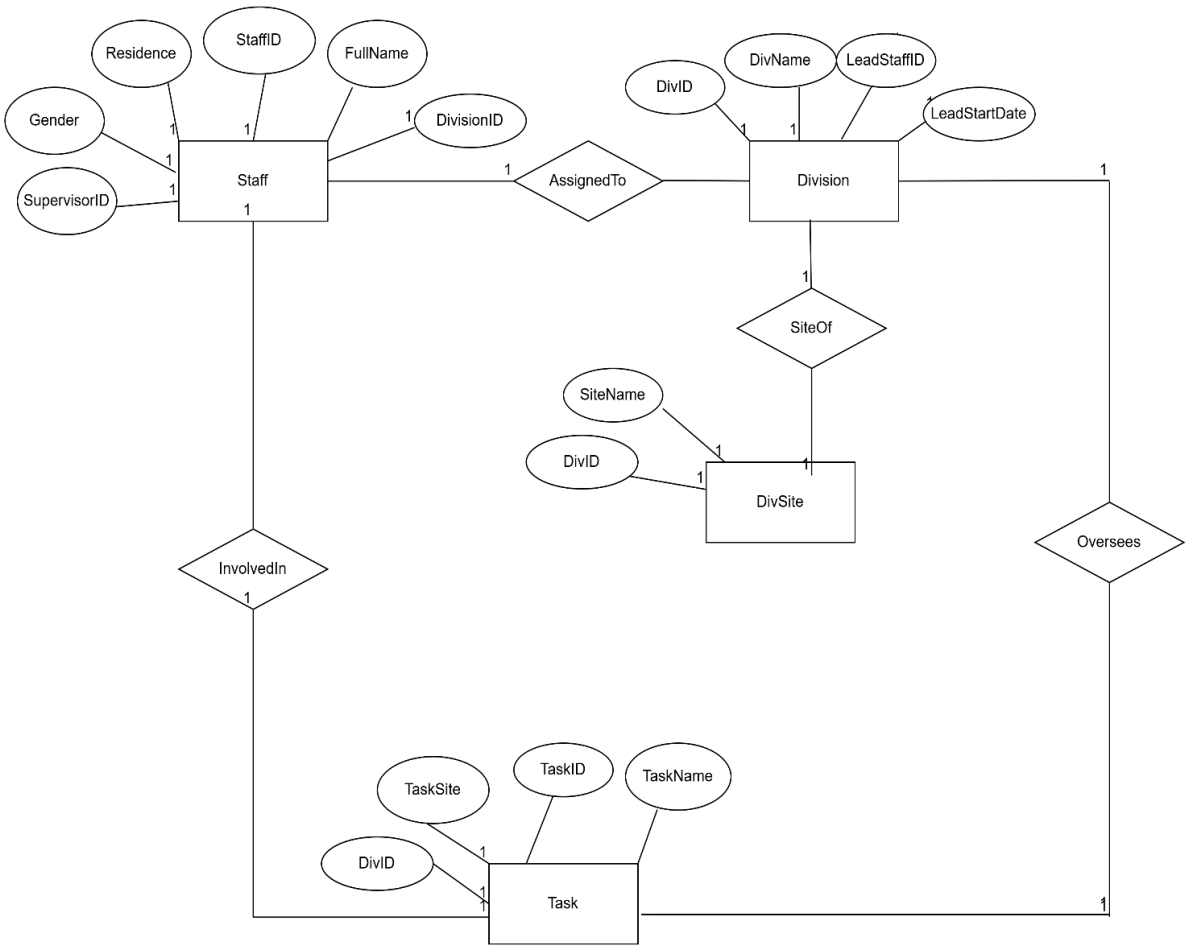
5. Assignment (staff-task relationship)

- StaffID
- TaskID
- WorkHours

- **Relationships**

1. AssignedTo (Staff to Division)
   Many-to-One: Many staff members work in one division.
2. Leads (Staff to Division)
   One-to-One: One staff member leads one division.
3. SiteOf (Division to DivSite)
   One-to-Many: One division can have multiple sites.
4. Oversees (Division to Task)
   One-to-Many: One division oversees multiple tasks.
5. InvolvedIn (Staff to Task)
   Many-to-Many: Staff members work on multiple tasks using the Assignment entity with WorkHours.

Result
The ER diagram represents the organizational structure and supports queries related to staff assignments, divisions, and tasks.



Experiment – 2

Aim : To design an Entity–Relationship (ER) diagram for a student academic management system and define the relationship types among entities.

Objectives

- To list entities and attributes for student records
- To map relationships among entities
- To specify relationship cardinalities

Entities and Attributes

1. Learner (enrolled students)

- LoginID (Primary Key)
- LName
- HomeAddress
- ContactNo
- Gender

2. TermGroup (semester groups)

- GroupID (Primary Key)
- Term
- GroupNo

3. Enrollment (learner enrollment details)

- EnrollID (Primary Key)
- LoginID

4. Module (academic modules)

- ModCode (Primary Key)
- ModTitle
- Credits
- Term

5. Assessment (internal assessment scores)

- AssessID (Primary Key)
- Quiz1
- Quiz2
- Quiz3
- TotalIA

Relationships

1. RegistersFor (Learner to Enrollment)
One-to-Many: One learner can have multiple enrollments.

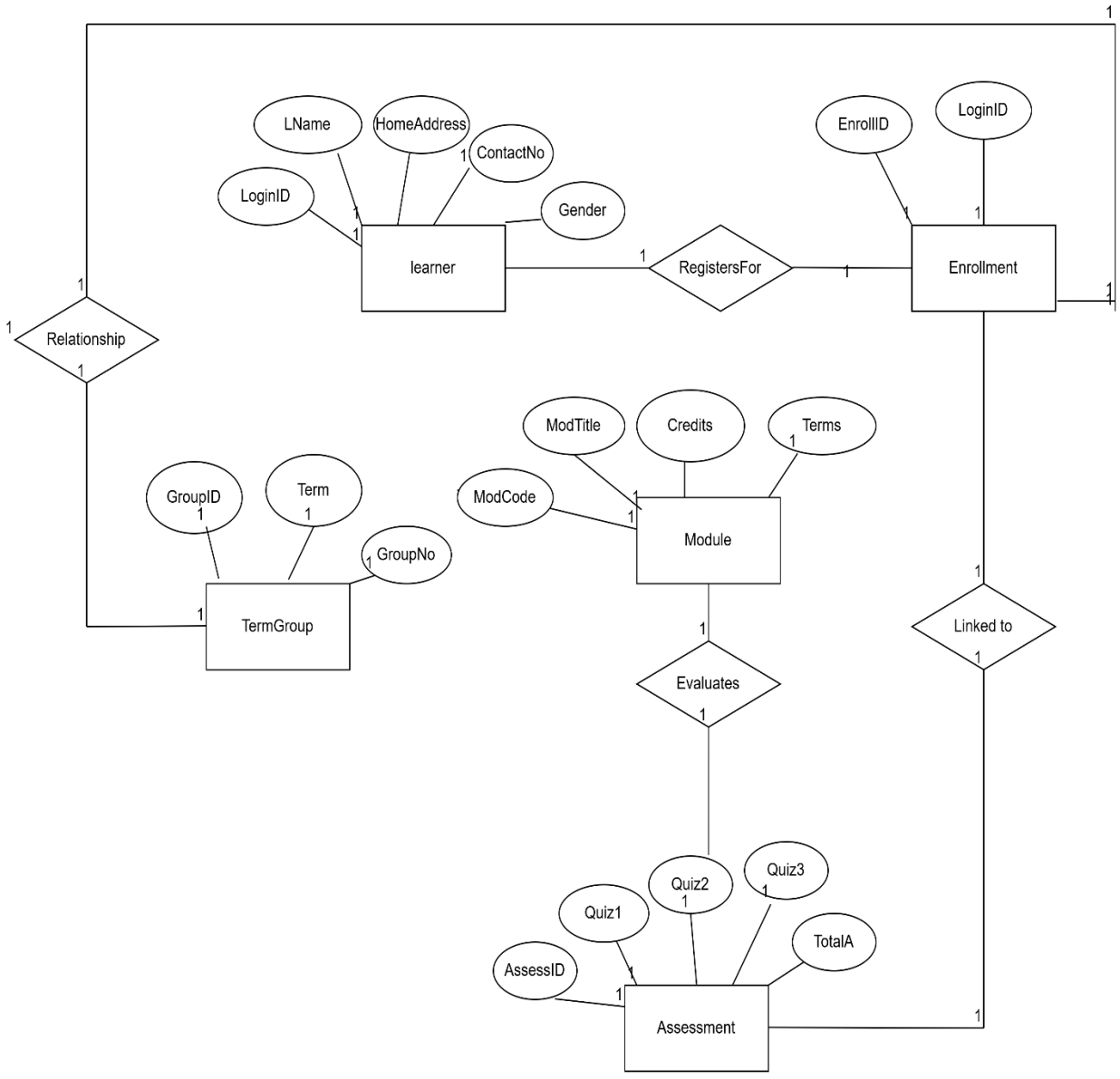
2. Includes (TermGroup to Enrollment)
One-to-Many: One term group includes many enrollments.

3. LinkedTo (Enrollment to Assessment)
One-to-One: Each enrollment has one assessment record.

4.Evaluates (Module to Assessment)
One-to-Many: One module has multiple assessments.

Result

This ER diagram helps in managing learner information, enrollments, modules, and assessment records efficiently.



EXPERIMENT-3

Assignment 3

Database Schema for a customer-sale scenario

Customer(Cust id : integer, cust_name: string)

Item(item_id: integer, item_name: string, price: integer)

Sale(bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)

For the above schema, perform the following—

1. Create the tables with the appropriate integrity constraints.
2. Insert around 10 records in each of the tables.
3. List all the bills for the current date with the customer names and item numbers.
4. List the total Bill details with the quantity sold, price of the item and the final amount.
5. List the details of the customer who have bought a product which has a price>200.
6. Give a count of how many products have been bought by each customer
7. Give a list of products bought by a customer having cust_id as 5.
8. List the item details which are sold as of today.
9. Create a view which lists out the bill_no, bill_date, cust_id, item_id, price, qty_sold, amount.
10. Create a view which lists the daily sales date wise for the last one week

Code:

```
CREATE DATABASE dolly;


USE dolly;
CREATE TABLE Customer (
    cust_id INT PRIMARY KEY,
    cust_name VARCHAR(50) NOT NULL
);



CREATE TABLE Item (
    item_id INT PRIMARY KEY,
    item_name VARCHAR(50) NOT NULL,
    price INT CHECK (price > 0)
);



CREATE TABLE Sale (
    bill_no INT PRIMARY KEY,
    bill_date DATE NOT NULL,
    cust_id INT,
    item_id INT,
    qty_sold INT CHECK (qty_sold > 0),
    FOREIGN KEY (cust_id) REFERENCES Customer(cust_id),
    FOREIGN KEY (item_id) REFERENCES Item(item_id)
);
```

INSERT INTO Customer VALUES

(1, 'dolly'),

(2, 'akansha'),

(3, 'priyanshi'),

(4, 'Sneha'),

(5, 'Karan'),

(6, 'Meera'),

(7, 'Arjun'),

(8, 'Priya'),

(9, 'Vikas'),

(10, 'Anjali');

INSERT INTO Item VALUES

(101, 'Pen', 20),

(102, 'Notebook', 80),

(103, 'Bag', 500),

(104, 'Shoes', 1200),

(105, 'Bottle', 150),

(106, 'Watch', 2500),

(107, 'Keyboard', 700),

(108, 'Mouse', 400),

(109, 'Book', 300),

(110, 'Headphones', 1500);

INSERT INTO Sale VALUES

(1, CURDATE(), 1, 103, 1),

(2, CURDATE(), 2, 104, 2),

(3, CURDATE(), 5, 106, 1),

(4, '2026-02-20', 3, 101, 5),

(5, '2026-02-21', 4, 102, 3),

(6, '2026-02-22', 6, 110, 1),

```
(7, CURDATE(), 7, 108, 2),
(8, '2026-02-19', 8, 105, 4),
(9, '2026-02-18', 9, 109, 2),
(10, CURDATE(), 10, 107, 1);


SELECT S.bill_no, C.cust_name, S.item_id
FROM Sale S
JOIN Customer C ON S.cust_id = C.cust_id
WHERE S.bill_date = CURDATE();


SELECT S.bill_no,
    C.cust_name,
    I.item_name,
    S.qty_sold,
    I.price,
    (S.qty_sold * I.price) AS final_amount
FROM Sale S
JOIN Customer C ON S.cust_id = C.cust_id
JOIN Item I ON S.item_id = I.item_id;


SELECT DISTINCT C.cust_id, C.cust_name
FROM Sale S
JOIN Customer C ON S.cust_id = C.cust_id
JOIN Item I ON S.item_id = I.item_id
WHERE I.price > 200;
```

```
SELECT C.cust_id, C.cust_name,
    COUNT(S.item_id) AS total_products
FROM Customer C
LEFT JOIN Sale S ON C.cust_id = S.cust_id
GROUP BY C.cust_id, C.cust_name;
```

```
SELECT I.item_id, I.item_name
FROM Sale S
JOIN Item I ON S.item_id = I.item_id
WHERE S.cust_id = 5;
```

```
SELECT DISTINCT I.*
FROM Sale S
JOIN Item I ON S.item_id = I.item_id
WHERE S.bill_date = CURDATE();
```

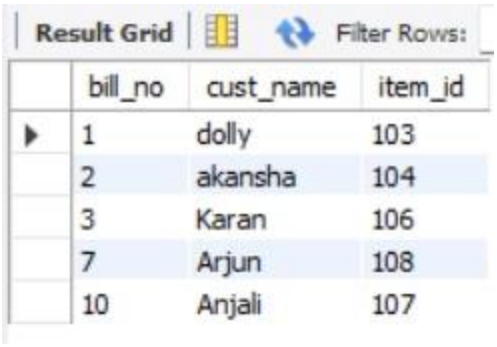
```
CREATE VIEW Bill_View AS
SELECT S.bill_no,
    S.bill_date,
    S.cust_id,
    S.item_id,
    I.price,
    S.qty_sold,
    (S.qty_sold * I.price) AS amount
```

```
FROM Sale S
JOIN Item I ON S.item_id = I.item_id;
```

```
CREATE VIEW Weekly_Sales AS
SELECT bill_date,
    SUM(S.qty_sold * I.price) AS total_sales
FROM Sale S
JOIN Item I ON S.item_id = I.item_id
WHERE bill_date >= CURDATE() - INTERVAL 7 DAY
GROUP BY bill_date;
```

Output:

. List All Bills for Current Date with Customer Names and Item Number



bill_no	cust_name	item_id
1	dolly	103
2	akansha	104
3	Karan	106
7	Arjun	108
10	Anjali	107

List Total Bill Details (Quantity, Price, Final Amount)

Result Grid | Filter Rows: | Export: | Wrap C

bill_no	cust_name	item_name	qty_sold	price	final_amount
1	dolly	Bag	1	500	500
2	akansha	Shoes	2	1200	2400
3	Karan	Watch	1	2500	2500
4	priyanshi	Pen	5	20	100
5	Sneha	Notebook	3	80	240
6	Meera	Headphones	1	1500	1500
7	Arjun	Mouse	2	400	800
8	Priya	Bottle	4	150	600
9	Vikas	Book	2	300	600
10	Anjali	Keyboard	1	700	700

.Customers Who Bought Product with Price > 200

Result Grid | Filter

cust_id	cust_name
1	dolly
2	akansha
5	Karan
10	Anjali
7	Arjun
9	Vikas
6	Meera

.Count of Products Bought by Each Customer

Result Grid | Filter Rows:

cust_id	cust_name	total_products
1	dolly	1
2	akansha	1
3	priyanshi	1
4	Sneha	1
5	Karan	1
6	Meera	1
7	Arjun	1
8	Priya	1
9	Vikas	1
10	Anjali	1

Item Details Sold Today

Result Grid | Filter Rows:

item_id	item_name	price
103	Bag	500
104	Shoes	1200
106	Watch	2500
108	Mouse	400
107	Keyboard	700

View for Daily Sales (Last One Week)

Result Grid | Filter Rows:

bill_date	total_sales
2026-02-28	6900
2026-02-21	240
2026-02-22	1500